

**SQL MATCH_RECOGNIZE for Pandas
DataFrames:
A Python Implementation of SQL:2016
Row Pattern Recognition**

Master's Thesis
[University Name]

Submitted by
MonierAshraf

[Year]

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Research Objectives and Contributions	4
1.3	Organisation of the Thesis	5
2	Background and Concepts	6
2.1	Pattern Matching in Sequential Data Analysis	6
2.1.1	Real-World Applications	7
2.2	The SQL:2016 MATCH_RECOGNIZE Clause	8
2.2.1	Clause Structure and Components	9
2.2.2	Illustrative Example: Stock Price V-Pattern	11
2.2.3	Comparison of Existing Implementations	12
3	Related Work	15
3.1	SQL Engines with Native Support	15
3.1.1	Oracle	16
3.1.2	Trino	16
3.1.3	Snowflake	16
3.1.4	Apache Flink	17
3.1.5	Bringing MATCH_RECOGNIZE to DuckDB	17
3.1.6	Implementation Restrictions Across Systems	17
3.2	Pandas and the Python Ecosystem Gap	19
3.3	Contribution of This Work	19
4	Methodology	21
4.1	SQL Parsing	22
4.1.1	What is Parsing?	22
4.1.2	Parsing SQL: Existing Solutions	23
4.1.3	Grammars	23
4.1.4	Our Parsing Needs	24
4.1.5	Parser Generators	26

4.1.6	Alternative Methods	27
4.1.7	ANTLR4	27
4.1.8	Implementation of the Parser	28
4.1.9	Visitor Implementation	30
4.1.10	Discussion of the Parser	31
4.1.11	Safe SQL Expression Evaluation	32
4.2	Theoretical Foundations: Finite Automata	33
4.2.1	Non-deterministic Finite Automata (NFA)	33
4.2.2	Thompson’s Construction	34
4.2.3	Pattern Tokenisation	34
4.2.4	NFA Construction from Tokens	36
4.2.5	Properties of the Constructed NFA	39
4.2.6	Worked Example: Pattern A B+ C?	40
4.2.7	Deterministic Finite Automata (DFA)	41
4.3	Pandas DataFrame Execution Model	42
4.4	Performance Optimization Techniques	43
4.4.1	LRU Pattern Caching	43
4.4.2	Memory Management	43
5	Experimental Evaluation	44
5.1	Evaluation Methodology	44
5.2	Dataset and Test Patterns	44
5.3	Correctness Results	44
5.4	Performance Results	44
5.5	Comparison with Existing Systems	44
6	Conclusions and Future Work	45
6.1	Summary of Contributions	45
6.2	Limitations	45
6.3	Future Work	45
	Bibliography	46

List of Figures

4.1	NFA for the pattern $A^+ B?$ illustrating ε -transitions and non-determinism	33
-----	--	----

List of Tables

2.1	Existing <code>MATCH_RECOGNIZE</code> implementations	13
3.1	Comparison of SQL engines providing <code>MATCH_RECOGNIZE</code> support . .	18
4.1	Comparison of Python-compatible parser generators	26
4.2	Principal AST node types	29
4.3	The thirteen <code>PatternTokenType</code> values and their roles in the pattern string	35
4.4	Token types and their NFA fragment builders in <code>NFABuilder._process_sequence()</code>	38
4.5	Pattern cache configuration defaults	43

List of Algorithms

1	Thompson's NFA Construction	34
---	---------------------------------------	----

Chapter 1

Introduction

Pattern matching is the process of analysing a sequence of data points—whether rows in a relational table or events in a data stream—with the goal of identifying specific shapes, trends, or behaviour sequences within the data. It plays a crucial role across many analytical domains: detecting chains of transactions associated with money laundering in financial systems [petkovic2022fraud], revealing multi-step fraudulent behaviour sequences in security event logs, identifying equipment degradation patterns in IoT sensor streams, and tracking multi-step user journeys in e-commerce clickstreams. Unlike conventional SQL predicates that evaluate each row in isolation, pattern matching takes the temporal ordering and relative values of successive rows into account. Patterns are described using a regular-expression-like syntax applied at the row level, enabling analysts to match and extract meaningful sequences across entire records.

1.1 Problem Statement

The `MATCH_RECOGNIZE` (MR) clause was introduced in the SQL:2016 standard [iso2016, ISO2021], enabling users to define and search for patterns within ordered sequences of rows directly in SQL queries. Although standardised nearly a decade ago, only a small number of database management systems have implemented it. Among established relational database systems, Oracle [oracle2022, book] was the first to provide a production-quality implementation (in Database 12c, 2013), conforming to Feature R010 of the standard. Several modern analytical platforms followed: Trino [trino2022, trino2021row], Snowflake [snowflake2025match__recognize], and Apache Flink [flink] each provide their own implementations, though with varying degrees of conformance to the SQL:2016 feature levels (R010, R020, R030) and with important practical restrictions. Notably, PostgreSQL and DuckDB—two widely used open-source systems—do not yet support `MATCH_RECOGNIZE` [howe2020matchrecognize, findling2025bringing].

Without this declarative clause, detecting multi-row patterns requires complex combinations of SQL window functions, self-joins, or procedural loops over rows outside the database. As the pattern grows beyond two or three steps, the corresponding code expands into dozens of lines filled with row-number calculations, timestamp comparisons, and cascading JOIN conditions. Such code is difficult to write, harder to maintain, and typically forces multiple table scans with poor performance [cao2012optimizationanalyticwindowfunctions].

In Python’s data science ecosystem, the limitation is even more acute. Pandas [7] is the dominant library for in-process tabular data analysis, yet it provides no native sequential pattern primitive. Analysts who need to detect multi-step patterns must manually compose low-level operations: creating multiple shifted column copies (`df['value'].shift(-1), shift(-2), ...`), building Boolean masks, stitching subsequences with `groupby().apply()`, and pivoting or renaming columns. Listing 1.1 illustrates this approach for a simple three-step decreasing-price pattern (A B C).

```
1 import pandas as pd
2
3 df['order_date'] = pd.to_datetime(df['order_date'])
4 df = df.sort_values(['customer_id', 'order_date']).
    reset_index(drop=True)
5
6 def match_pattern_ABC(group: pd.DataFrame):
7     matches = []
8     for i in range(len(group) - 2):
9         A, B, C = group.iloc[i], group.iloc[i+1], group.iloc[
10             i+2]
11         if (B['price'] < A['price']) and (C['price'] < B['
12             price']):
13             matches.append({
14                 'customer_id': A['customer_id'],
15                 'start_date': A['order_date'],
16                 'end_date': C['order_date'],
17                 'bottom_price': C['price']
18             })
19     return matches
20
21 all_matches = []
22 for _, grp in df.groupby('customer_id'):
23     all_matches.extend(match_pattern_ABC(grp))
24
25 result_df = pd.DataFrame(all_matches)
```


Listing 1.1: Detecting a three-step price-decrease pattern in Pandas *without* MATCH_RECOGNIZE

This hand-crafted approach is verbose (over 20 lines for a three-step pattern), prone to off-by-one alignment errors, and does not scale gracefully to longer or conditional patterns. The same query expressed with MATCH_RECOGNIZE is shown in Listing 1.2: a single declarative statement that specifies the partition, ordering, pattern grammar, variable definitions, and output measures.

```
1 import pandas as pd
2 from src.executor.match_recognize import match_recognize
3
4 df['order_date'] = pd.to_datetime(df['order_date'])
5
6 query = """
7 SELECT customer_id, start_date, end_date, bottom_price
8 FROM orders
9 MATCH_RECOGNIZE (
10     PARTITION BY customer_id
11     ORDER BY      order_date
12     MEASURES
13         A.order_date AS start_date,
14         C.order_date AS end_date,
15         C.price       AS bottom_price
16     ONE ROW PER MATCH
17     AFTER MATCH SKIP PAST LAST ROW
18     PATTERN (A B C)
19     DEFINE
20         B AS B.price < A.price,
21         C AS C.price < B.price
22 )
23 """
24
25 output_df = match_recognize(query, df)
```

Listing 1.2: The same pattern expressed with MATCH_RECOGNIZE on a Pandas DataFrame

The declarative form is not merely shorter syntactically: it separates *what* to find (the pattern) from *how* to find it (the engine), making patterns composable, auditable, and easy to extend. A more complex pattern—say, four or five steps with

conditional alternation—requires no change to the surrounding Python code, only an update to the `PATTERN` and `DEFINE` clauses [michels2018new, petkovic2022rpr]. This approach also enables the engine to apply automata-based optimisations—LRU pattern caching, partition-wise memory management, and NFA/DFA execution—transparently.

The full structure and sub-clauses of `MATCH_RECOGNIZE` are introduced in Chapter 2. Chapter 3 surveys the existing implementations and identifies the research gap.

1.2 Research Objectives and Contributions

This thesis closes the gap between the SQL:2016 `MATCH_RECOGNIZE` standard and Python’s Pandas ecosystem, bridging the expressiveness of relational pattern queries with the flexibility of in-memory analytics. Three objectives guided the work; each is stated below together with the contribution that fulfils it.

1. **An in-process, SQL:2016-compliant engine for Pandas DataFrames.**

Without a declarative alternative in Python, a data scientist must write the twenty-line hand-crafted loop in Listing 1.1 merely to detect a simple three-step declining-price pattern. This thesis delivers the **first SQL:2016 Feature R010-compliant `MATCH_RECOGNIZE` implementation** that runs entirely in-process alongside Pandas—no external database server or cloud subscription required—reducing that loop to the single declarative call in Listing 1.2 and eliminating the need for complex Python code or external SQL engine dependencies.

2. **Full coverage of SQL:2016 pattern constructs.** The engine supports the complete construct set defined by the standard: quantifiers (`*`, `+`, `?`, `{n,m}`), alternation (`|`), grouping, anchors, exclusions (`{- -}`), and `PERMUTE`. For example, a fraud analyst can express a probe-and-drain sequence as

```
PATTERN (PROBE+ (TRANSFER | WITHDRAW){2,5} DRAIN)
```

and the engine parses, compiles, and executes it correctly. Correctness is validated across 25 diverse test patterns in Chapter 5. One acknowledged boundary: patterns with three or more levels of nested greedy quantifiers (e.g., `(A+B*)+C?`) can trigger exponential automata state-space growth; bounded quantifiers and simpler nesting behave efficiently [michels2018new].

3. **Safe predicate evaluation and empirical validation at real-world scale.**

User-written `DEFINE` predicates must be evaluated *securely*: passing strings

directly to Python’s `eval()` is a code-injection vulnerability (a predicate such as `__import__('os').system('rm /')` would execute arbitrary system commands). The engine instead parses predicates into an AST and evaluates them through a controlled interpreter that whitelists permitted operations. Correctness and performance are validated on the Amazon UK Products 2023 catalogue [asaniczka2023amazon] (2.2 million rows): 100% correctness across all 25 test patterns and throughput up to **12 918 rows/sec** on a commodity laptop. Future work will address performance on larger datasets, memory consumption for high-variable patterns, and integration with query-optimiser plan-level improvements.

1.3 Organisation of the Thesis

The remainder of this thesis is organised as follows.

Chapter 2 — Background and Concepts presents the theoretical and technical foundations on which this work is built. It introduces sequential pattern matching and its real-world applications, explains the SQL:2016 `MATCH_RECOGNIZE` clause and its eight sub-clauses in detail, and provides an overview of existing implementations together with the feature-level classification (R010, R020, R030) used throughout the thesis.

Chapter 3 — Related Work surveys each major `MATCH_RECOGNIZE` implementation in depth—Oracle, Trino, Snowflake, Apache Flink, and DuckDB—documenting their feature levels, execution strategies, and practical restrictions. It also examines the Python and Pandas ecosystem gap and formally positions the contribution of this thesis within the broader pattern-matching research landscape.

Chapter 4 — Methodology describes the technical approach: finite automata (NFA construction via Thompson’s algorithm, subset-construction DFA, and minimisation), the compiler pipeline (ANTLR4 grammar, AST construction, safe expression evaluation), the Pandas execution model, and the performance optimisation techniques.

Chapter 5 — Experimental Evaluation presents the evaluation methodology, the benchmark dataset (Amazon UK Products 2023, 2.2 million rows), and experimental results covering correctness across 25 test patterns, performance scalability, pattern complexity, and comparison with existing systems.

Chapter 6 — Conclusions and Future Work summarises the main contributions, reflects on the limitations of the current implementation, and outlines directions for future research and further extensions.

Chapter 2

Background and Concepts

This chapter establishes the theoretical and practical foundations required to understand the design and implementation of the `MATCH_RECOGNIZE` engine for Pandas DataFrames. Section 2.1 introduces the problem of sequential pattern detection in relational data. Section 2.2 examines the SQL:2016 `MATCH_RECOGNIZE` clause in detail. Section 4.4 in Chapter 4 describes performance optimization techniques. Chapter 3 surveys related work and identifies the research gap addressed by this thesis.

2.1 Pattern Matching in Sequential Data Analysis

Sequential pattern detection represents a fundamental challenge in data analysis, where the objective is to identify ordered sequences of events or states within large datasets. In contrast to conventional queries that match rows individually against one or more predicates, pattern matching takes preceding and subsequent rows into consideration and returns groups of rows when they follow a specified pattern. This makes it possible to retrieve specific shapes—such as V-, W- or U-shapes—within the data, making pattern matching predestined for all kinds of processes driven by sequences of events: detecting unusual behaviour in security applications, analysing stock price development, sensor data analysis, fraud detection, and generally complex event processing [2].

Unlike traditional relational operations that focus on individual rows or aggregated values, pattern matching examines the temporal or logical ordering of data points to extract meaningful sequences. This capability is essential across many domains including financial market analysis, fraud detection, IoT sensor monitoring, and behavioural analytics.

Historically, relational database systems excelled at set-based operations—selection, projection, joins, and aggregation—but lacked expressive mechanisms for sequence analysis. Analysts requiring pattern detection were forced to either export data to

specialised stream-processing systems, implement complex procedural logic using cursors and temporary tables, or employ application-layer processing with significant performance overhead. These limitations created a gap between the widespread availability of sequential data and the tools available to analyse it effectively. The introduction of row pattern recognition in SQL:2016 addresses this deficiency by bringing sequence analysis capabilities directly into the query language [1].

2.1.1 Real-World Applications

Financial Market Analysis. Detecting V-shaped price patterns—price decline followed by recovery—requires examining ordered sequences of price movements. For example, identifying stocks whose price troughs are successively higher (a “higher-low” pattern) signals a potential upward trend. The canonical example used throughout this thesis is the detection of such V-patterns in customer order data [2].

Fraud Detection. Credit card fraud often manifests as specific transaction sequences, such as multiple small test charges followed by a large purchase, or geographically impossible transactions occurring within unrealistic timeframes. Expressing these rules declaratively in SQL is more maintainable than equivalent procedural logic.

IoT and Sensor Data. Manufacturing systems monitor sensor readings to detect equipment degradation patterns, such as a gradual temperature rise followed by a sudden drop, which may indicate imminent failure. Pattern queries over time-series sensor streams are a natural fit for `MATCH_RECOGNIZE`.

Clickstream and Funnel Analysis. E-commerce platforms analyse user navigation sequences to identify conversion paths or abandonment patterns. A classic funnel query detects the sequence “view → add-to-cart → remove → exit,” which signals purchase hesitation. With `MATCH_RECOGNIZE`, the analyst can also quantify *how many* add-to-cart events occurred before removal, extract the exact timestamps of each step, and compute the total dwell time in a single declarative query—tasks that would otherwise require multiple correlated subqueries or procedural session-stitching logic.

Network Security and Intrusion Detection. Cyber-attacks rarely manifest as a single anomalous event; they unfold as ordered sequences of actions. A typical multi-stage intrusion follows the pattern: port scanning → repeated failed authentication attempts → a single successful login → privilege escalation. `MATCH_RECOGNIZE` can express this entire kill-chain as one pattern query over an event log, flagging the

offending session and extracting the first and last timestamps, the targeted port, and the account compromised—all without exporting the log to a separate SIEM tool [petkovic2022fraud].

Healthcare and Patient Monitoring. In clinical data streams, early warning systems must detect physiological deterioration before a critical event. A typical sepsis-onset pattern consists of rising heart rate rows, followed by a drop in blood pressure rows, followed by a spike in white-blood-cell count. Expressing this over an ordered time-series of vital signs with `MATCH_RECOGNIZE` allows hospitals to produce real-time alerts from the same SQL infrastructure used for reporting, without duplicating data into a separate stream-processing system.

Supply Chain and Logistics. Operational disruptions in supply chains often emerge as a cascade: a single late shipment → repeated partial deliveries → a stockout event. `MATCH_RECOGNIZE` applied to an order fulfilment log can identify such escalation sequences per supplier or warehouse and measure the time between first delay and eventual stockout, enabling proactive mitigation rather than retrospective reporting.

Financial Compliance and Trade Surveillance. Regulatory frameworks such as MiFID II require exchanges to detect potential market manipulation patterns, for example “layering,” where a trader places a large resting order (to move the price), then executes a trade on the opposite side, then cancels the resting order. This three-step sequence—place, trade, cancel—with precise timing constraints is naturally expressed as a `MATCH_RECOGNIZE` pattern, turning a complex compliance rule directly into an auditable SQL query.

2.2 The SQL:2016 `MATCH_RECOGNIZE` Clause

Nearly a decade ago, the SQL:2016 standard introduced `MATCH_RECOGNIZE` [iso2016, 1], a construct which enabled row pattern matching in SQL. Row pattern matching is a technique which allows finding sequences of rows based on a pattern defined by regular expressions. Using a syntax similar to regular expressions, it is possible to filter data sets processed by pattern specifications to recognise complex sequences of events [winand2017sqlnew]. This process facilitates the recognition of patterns in data series that would otherwise require the use of additional SQL clauses, such as the `WINDOW` clause. Consequently, the `MATCH_RECOGNIZE` clause does not introduce new data-filtering power but rather serves as a shortcut to simplify special areas of application that can otherwise be circumvented by combining other clauses.

Up to now, `MATCH_RECOGNIZE` is not supported by many database management systems, including PostgreSQL [howe2020matchrecognize] and DuckDB [findling2025bringing]. This thesis contributes to the overarching research topic of making pattern matching available to DBMSs that do not natively support it, so that a wider range of users can benefit from its power and expressiveness. The clause brings capabilities previously available only in specialised stream-processing systems into standard relational databases.

The SQL standard distinguishes between different feature levels for row pattern matching, which can be used to classify implementations [winand__match_recognize]. **Feature R010** provides the most basic support and serves as a basis for further features; it includes row pattern recognition inside the `FROM` clause and basic aggregate functions (`MIN`, `MAX`, `SUM`, `COUNT`, `AVG`) inside the `MEASURES` clause. **Feature R020** extends the areas of application: `MATCH_RECOGNIZE` can be used within the `WINDOW` and `OVER` clauses to describe windows more precisely. **Feature R030** allows full aggregate support inside the `MATCH_RECOGNIZE` clause [ISO2021, michels2018new, melton2017sqlrpr].

2.2.1 Clause Structure and Components

The clause operates on a rowset (table or derived table) and consists of eight interconnected sub-clauses, as shown in Listing 2.1.

```

1  SELECT <columns>
2  FROM   <table>
3  MATCH_RECOGNIZE (
4      PARTITION BY <partition_columns>      -- 1. Data
         organisation
5      ORDER BY   <ordering_columns>         -- 2. Row sequencing
6      MEASURES   <expressions>              -- 3. Output
         computations
7      <rows_per_match>                       -- 4. Result
         granularity
8      AFTER MATCH SKIP <option>              -- 5. Overlap handling
9      PATTERN ( <pattern_regex> )           -- 6. Sequence
         specification
10     SUBSET    <union_variable_defs>        -- 7. Union variables
         (optional)
11     DEFINE   <variable_definitions>      -- 8. Variable
         predicates
12 ) AS <alias>

```

Listing 2.1: General syntax of the `MATCH_RECOGNIZE` clause

1. **PARTITION BY.** Divides the input rowset into independent partitions, analogous to window-function partitioning. Pattern matching runs independently within each partition, enabling both parallel processing and domain-specific grouping (e.g. per customer or per sensor).
2. **ORDER BY.** Establishes the logical ordering of rows within each partition. The pattern-matching algorithm processes rows strictly according to this ordering. Multiple sort keys with `ASC/ DESC` modifiers are supported.
3. **MEASURES.** Defines computed expressions that extract values from matched row sequences using pattern variables and navigation functions (`FIRST`, `LAST`, `PREV`, `NEXT`). Each `MEASURE` expression becomes a column in the output.
4. **Output options.**
 - `ONE ROW PER MATCH`: one summary row per complete match.
 - `ALL ROWS PER MATCH`: one row for every row participating in a match.
 - `ALL ROWS PER MATCH WITH UNMATCHED ROWS`: includes non-matching rows in the result.
5. **AFTER MATCH SKIP.** Controls how the engine resumes after finding a match:
 - `SKIP TO NEXT ROW` – resume from the row after match start (maximum overlaps).
 - `SKIP PAST LAST ROW` – resume from the row after match end (non-overlapping matches).
 - `SKIP TO FIRST var` – resume from the first row classified as *var*.
 - `SKIP TO LAST var` – resume from the last row classified as *var*.
6. **PATTERN.** Specifies the sequence using regular-expression-like syntax with pattern variables as terminals:
 - Concatenation: `A B C` (A followed by B followed by C).
 - Alternation: `A | B` (either A or B).
 - Quantifiers: `A*` (zero or more), `A+` (one or more), `A?` (zero or one), `A{n}` (exactly *n*), `A{n,m}` (between *n* and *m*).

- Grouping: (A B)+ (one or more AB pairs).
 - Anchors: ^A B C\$ (pattern must span the entire partition).
 - Exclusions: {- A+ B+ -} (rows *not* matching the enclosed sub-pattern).
 - Permutations: PERMUTE(A, B, C) (any ordering of A, B, and C).
7. **SUBSET.** An optional clause for defining union variables, each of which consists of a set of variables defined in the **PATTERN** clause. Union variables can be used to match rows to one of several different patterns, and may appear inside **MEASURES** and **DEFINE** expressions.
8. **DEFINE.** Assigns Boolean predicates to pattern variables. The initial step in execution involves implementing a filter on the ordered data using this clause, which specifies a list of named Boolean expressions (*steps*) that either enable or deny the inclusion of a row within the matched sequence based on a predetermined criterion. Each row is evaluated against these predicates to determine its variable classification. Predicates may reference current-row columns, navigation functions, aggregations over matched sub-sequences, and Boolean/comparison operators [**winand__match__recognize**].

The complete **MATCH_RECOGNIZE** clause is embedded in a **SELECT** query and can therefore be further processed by additional specifications. Possible extensions include applying groupings or aggregate functions to the outer **SELECT** [**winand__match__recognize**].

2.2.2 Illustrative Example: Stock Price V-Pattern

Listing 2.2 shows a complete **MATCH_RECOGNIZE** query that detects V-shaped price patterns in customer order data.

```

1  SELECT customer_id,
2         start_price, bottom_price, final_price,
3         start_date,  final_date
4  FROM   orders
5  MATCH_RECOGNIZE (
6      PARTITION BY customer_id
7      ORDER BY    order_date
8      MEASURES
9          START.price           AS start_price,
10         LAST(DOWN.price)      AS bottom_price,
11         LAST(UP.price)        AS final_price,
12         START.order_date      AS start_date,
13         LAST(UP.order_date)   AS final_date

```

```

14  ONE ROW PER MATCH
15  AFTER MATCH SKIP PAST LAST ROW
16  PATTERN (START DOWN+ UP+)
17  DEFINE
18      DOWN AS price < PREV(price),
19      UP   AS price > PREV(price)
20 );

```

Listing 2.2: Detecting V-shaped price patterns with `MATCH_RECOGNIZE`

The engine partitions rows by `customer_id`, sorts by `order_date`, and seeks sequences matching `START DOWN+ UP+`: one `START` row (any row, as it carries no `DEFINE` predicate), one or more `DOWN` rows (each with a strictly lower price than its predecessor), and one or more `UP` rows (each with a strictly higher price than its predecessor). For every match found, the `MEASURES` sub-clause extracts the starting price and date from the `START` row, the lowest price from the last `DOWN` row, and the final price from the last `UP` row.

2.2.3 Comparison of Existing Implementations

Several database and stream-processing systems have implemented `MATCH_RECOGNIZE`. The SQL standard defines three feature profiles for classifying those implementations [ISO2021, michels2018new]: **Feature R010** covers row pattern recognition in the `FROM` clause together with basic aggregate functions; **Feature R020** additionally permits `MATCH_RECOGNIZE` inside `WINDOW/OVER` clauses; and **Feature R030** enables full aggregate support within the clause. In practice, most systems have adopted only Feature R010 as the baseline. Snowflake is the only publicly available engine that additionally implements Feature R020. Feature R030 has not been adopted by any major production system to date.

The following paragraphs briefly characterise each implementation included in Table 2.1.

Oracle introduced `MATCH_RECOGNIZE` in Database 12c (2013) and remains the primary reference implementation [oracle2022, oracle2021dwhsg, book]. It implements Feature R010 using an NFA-based engine with backtracking optimisations for complex patterns. Feature R020 is not supported; the clause cannot appear inside a `WINDOW` function. Oracle also enforces several additional restrictions: Boolean expressions are not permitted in the `MEASURES` clause, and `DISTINCT` may not precede aggregate functions. Despite its technical depth, Oracle operates under a commercial licence, making it inaccessible for many users.

Trino is an open-source distributed query engine whose `MATCH_RECOGNIZE` implementation is DFA-based, guaranteeing deterministic pattern matching in linear

time without backtracking [trino2022, trino2021row]. It supports Feature R010 only—the clause may only appear in the `FROM` clause. Trino’s architecture is designed for parallelised, federated query execution across large clusters, which differentiates it from single-node systems.

Snowflake is a cloud-native data platform that covers both Feature R010 and Feature R020 [snowflake2025match_recognize, hoffa2021snowflake]. In contrast to the SQL standard, Snowflake makes no syntactic distinction between the `FROM` and `WINDOW` usage contexts: the two are syntactically equivalent, and Feature R020 is realised purely through semantic interpretation. Snowflake charges by compute time (warehouse size), so query costs scale with workload.

Apache Flink targets real-time stream processing and implements `MATCH_RECOGNIZE` through its Complex Event Processing (CEP) library rather than as a native SQL sub-clause [vsprem2024building, georgiou2025streaming]. This makes Flink highly effective for high-velocity continuous data, but it introduces a separate programming model that is less familiar to analysts accustomed to batch SQL.

DuckDB is an embeddable, in-process analytical database that does not yet provide native `MATCH_RECOGNIZE` support [raasveldt2019duckdb, findling2025bringing]. Because DuckDB requires no separate server and exposes a Python API, it is architecturally close to this thesis’s target environment. Ongoing research investigates transpiling `MATCH_RECOGNIZE` to `WITH RECURSIVE` inside DuckDB as a path toward Feature R010 compliance.

Table 2.1 summarises these five systems alongside this work.

Table 2.1: Existing `MATCH_RECOGNIZE` implementations

Engine	Feature Level	Open Source	Deployment	Licence
Oracle	R010 (no R020)	No	On-premise / Cloud	Commercial
Trino	R010	Yes	Distributed	Apache 2.0
Snowflake	R010 + R020	No	Cloud-native	Commercial
Apache Flink	CEP library	Yes	Streaming cluster	Apache 2.0
DuckDB	None (planned)	Yes	In-process	MIT
This work	R010 (full)	Yes	In-process Python	Open-source

Two structural gaps are visible from this overview. First, every system with native `MATCH_RECOGNIZE` support—Oracle, Trino, Snowflake, and Flink—requires a separate server, a distributed cluster, or a paid cloud subscription, making them unsuitable for lightweight or in-process data science. Second, none of these systems integrates with Python’s Pandas ecosystem, which is the dominant platform for in-process analytical workloads. This thesis addresses both gaps by delivering a full Feature R010-compliant `MATCH_RECOGNIZE` engine that runs entirely inside a Pandas workflow with zero additional deployment.

Chapter 3 examines each of these systems in greater depth, documenting the precise restrictions each imposes, comparing their execution strategies, and situating the contribution of this work within the broader landscape of pattern-matching research.

Chapter 3

Related Work

Row pattern recognition was first proposed as a SQL standard feature by Zemke et al. [zemke2007pattern] and was officially adopted in SQL:2016 [iso2016, 1]. Subsequent editions refined and documented its semantics [ISO2021, melton2017sqlrpr, michels2018new]. The clause has since been studied both as a specification and in terms of its practical implementations [petkovic2022rpr, petkovic2022fraud].

The `MATCH_RECOGNIZE` clause is supported natively by many leading SQL engines, including Oracle [oracle2022, book], Snowflake [snowflake2025match__recognize], and Trino [trino2022, trino2021row]. Other systems are still considering adding this feature, including PostgreSQL [howe2020matchrecognize], MariaDB [mariadb2025regex], and Microsoft SQL Server [Zhu2023rowpattern]. To use `MATCH_RECOGNIZE` in systems without native support, users must emulate row-pattern matching using window functions, user-defined aggregates, or external procedural layers [TURP2, TURP, cao2012optimizationanalyticwindowfunctions]—approaches that are often more complex, less efficient, and harder to maintain.

3.1 SQL Engines with Native Support

Oracle [oracle2022, book], Trino [trino2022, trino2021row], Snowflake [snowflake2025match__recognize], and Flink [flink] all provide advanced solutions for SQL-based pattern matching, but they each come with important practical challenges that users need to keep in mind. Although `MATCH_RECOGNIZE` was already defined as part of the SQL:2016 standard, it has only been implemented in a limited number of DBMSs to date [petkovic2022rpr]. Given the construct’s capacity to encapsulate the use of numerous other queries as a simplification, it would be beneficial to extend its application to more database systems. The Oracle implementation serves as the primary reference point for conformance with the SQL standard [oracle2022, oracle2021dwhsg, book].

3.1.1 Oracle

Oracle has been a pioneer in SQL `MATCH_RECOGNIZE`, having introduced row pattern recognition in Version 12c. It provides a comprehensive and robust implementation well-suited for complex, enterprise-scale analytics, implementing `MATCH_RECOGNIZE` based on Feature R010. It executes pattern-matching queries in a highly efficient manner through internal optimisations, including backtracking algorithms specifically designed to address complex patterns. Due to implementation constraints, however, Oracle does not support Feature R020—that is, the use of `MATCH_RECOGNIZE` within the `WINDOW` clause is not permitted. There are also further restrictions during use: for example, Boolean expressions are not allowed in the `MEASURES` clause, and the `DISTINCT` keyword cannot be used before aggregates. Despite its technical capability, Oracle’s power comes with high operational costs, including commercial licensing fees, which makes it less accessible for many users [oracle2022, oracle2021dwhsg, book].

3.1.2 Trino

As an open-source project, Trino is defined by its role as a distributed data analytics tool offering capabilities for exploring voluminous data sets, where only a limited number of competitors provide analogous functionality within the same scope. In contrast to other DBMSs that are predominantly single-node, Trino is engineered for parallel processing, thereby enabling management of larger data sets. The Trino `MATCH_RECOGNIZE` implementation is DFA-based, a distinction that enables deterministic pattern matching in linear time—a key difference from other engines that may rely on backtracking. However, `MATCH_RECOGNIZE` can only be used inside the `FROM` clause, thereby implementing only Feature R010. It is more accessible than Oracle in terms of deployment and cost, but still requires considerable infrastructure management and may lack some of Oracle’s advanced optimisations [trino2022, trino2021row].

3.1.3 Snowflake

Snowflake is a prominent cloud-native platform that offers `MATCH_RECOGNIZE` support within its own syntax. It supports the `MATCH_RECOGNIZE` clause both in the `FROM` clause and within the `WINDOW` clause, covering Features R010 and R020. However, in contrast to the SQL standard, Snowflake makes no syntactic distinction between these two application areas: a `WINDOW` clause containing `MATCH_RECOGNIZE` exhibits a strong similarity to its use in `FROM`, so Feature R020 is implemented only semantically rather than through an explicit syntactic separation. Snowflake also offers limited

support for advanced clauses and has computational costs that scale with warehouse size [snowflake2025match_recognize, hoffa2021snowflake].

3.1.4 Apache Flink

Apache Flink [vsprem2024building, georgiou2025streaming] is designed for real-time stream processing, allowing rapid detection of event patterns in continuous data flows. While Flink is highly scalable and effective for high-velocity data, it introduces its own learning curve and is less familiar to users accustomed to traditional, batch-based SQL systems. A major limitation shared by all these platforms is the complexity of integrating SQL-based pattern matching with machine learning workflows and other analytics tools [ML_Workflow, Makrynioti2019sql4mlAD]. Typically, data must be exported from the database to external environments for ML modelling, resulting in data duplication, consistency challenges, and more complicated data pipelines [naphade2025evolution].

3.1.5 Bringing MATCH_RECOGNIZE to DuckDB

DuckDB is a relational database management system that offers several advantages over the DBMSs discussed above [raasveldt2019duckdb]. The most significant advantage is that it does not require a separate server for operation; instead it can be embedded directly into the host application, making it well-suited for small projects and in-process analytics. DuckDB is open-source, enabling users to contribute new functionality, and it provides a Python API that permits the execution of DuckDB queries inside a transpiler for testing and debugging purposes [raasveldt2019duckdb].

Although MATCH_RECOGNIZE has not yet been implemented in DuckDB natively, DuckDB’s aim for completeness of the SQL interface means it supports a wide range of other clauses—including the WINDOW clause and WITH RECURSIVE—which can be leveraged to transpile MATCH_RECOGNIZE queries through a rule-based transformation approach [findling2025bringing, mogensen2024compiler, TURP2, TURP]. This approach filters multiple CTEs, thereby facilitating the different clauses contained within a MATCH_RECOGNIZE call. Any such implementation targeting DuckDB would aim for Feature R010 of the SQL standard, restricting row pattern matching to the FROM clause while still supporting a wide range of aggregate functions inside the MEASURES and DEFINE clauses.

3.1.6 Implementation Restrictions Across Systems

Every implementation of the MATCH_RECOGNIZE clause has certain restrictions that impose limitations during use. Common restrictions shared across systems [oracle2022,

`oracle2021dwhsg`, `trino2022`, `petkovic2022rpr`] include:

- The aggregate function `MATCH_NUMBER()` is not supported in the `MEASURES` or `DEFINE` clauses in several implementations.
- Column access as parameters within aggregate and navigation functions is the only permitted form, which limits the expressiveness of aggregate functions.
- The `DISTINCT` keyword before aggregates is unsupported in Oracle and related systems.
- Boolean expressions cannot be used directly in the `MEASURES` clause in Oracle.

For many users—especially those with modest analytics needs or who work independently—installing and managing large-scale systems like Oracle, Trino, Snowflake, or Flink can be overly complicated for some users. These users may only need to perform occasional pattern detection and would prefer to avoid the overhead of learning different functions or dealing with varying syntax for similar queries across platforms [8509400].

Table 3.1 summarises the key characteristics and feature-level support of these implementations.

Table 3.1: Comparison of SQL engines providing `MATCH_RECOGNIZE` support

Engine	Feature Level	Open Source	Deployment	Licence
Oracle	R010 (no R020)	No	On-premise / Cloud	Commercial
Trino	R010	Yes	Distributed	Apache 2.0
Snowflake	R010 + R020	No	Cloud-native	Commercial
Apache Flink	CEP library	Yes	Streaming cluster	Apache 2.0
DuckDB	None (planned)	Yes	In-process	MIT
Pandas (this work)	R010 (full)	Yes	In-process	Open-source

Additionally, for datasets that fit comfortably in memory, tools like Pandas offer a faster and more direct alternative, eliminating the latency of disk-based operations common to larger database engines [Evaluation, in-memory__processing]. Although Oracle, Trino, Snowflake, and Flink each have unique strengths for SQL pattern matching in large-scale or real-time scenarios, they introduce extra costs, infrastructure requirements, and integration issues that can be prohibitive for everyday users. These limitations highlight the value of Pandas, as it offers simpler, memory-based tools and unified workflows, where pattern matching and data analytics can be performed seamlessly and without unnecessary duplication or complexity.

3.2 Pandas and the Python Ecosystem Gap

Pandas is a popular, open-source Python library for data analysis and manipulation. Its straightforward installation process and strong community support make it a good choice for students, researchers, and professionals who frequently need to analyse data without deploying complicated systems like Oracle, Trino, or Snowflake. For everyday tasks, Pandas offers straightforward methods for managing data sequences, filtering, grouping, and performing basic time-series analyses.

However, Pandas has difficulty with advanced pattern detection in contrast to SQL’s `MATCH_RECOGNIZE` clause. It lacks a straightforward way to express complex patterns; users must create their own solutions using functions like `rolling`, `groupby`, or `shift`. These custom approaches can become complicated and error-prone, especially with advanced patterns.

Pandas is highly popular due to ease of setup and use, absence of licensing fees, and smooth integration within the larger Python libraries. This makes it great for quickly developing and testing ideas for typical pattern detection needs.

Despite Python’s dominance in data science, no maintained library implements SQL:2016 `MATCH_RECOGNIZE` semantics for Pandas DataFrames. Existing approaches require analysts to manually implement stateful logic using `DataFrame.apply()` chains, `iterrows()`, or external database round-trips—all of which are error-prone, verbose, and difficult to maintain. Alternative DataFrame libraries—Dask, Polars, Modin—similarly lack sequential pattern primitives. Even DuckDB, which recently added partial support for window analytics [findling2025bringing], does not yet expose a full `MATCH_RECOGNIZE` interface.

3.3 Contribution of This Work

This thesis addresses the identified gap with the following contributions:

1. The **first SQL:2016-compliant `MATCH_RECOGNIZE` implementation** that runs entirely in-process within a Python Pandas workflow with zero additional deployment.
2. **Complete pattern construct support**, including quantifiers, alternation, grouping, anchors, exclusions, and `PERMUTE`, faithfully following the SQL:2016 standard.
3. A **production-ready optimisation layer** comprising LRU pattern caching, partition-wise memory management, and thread-safe concurrent execution.

4. A **safe expression evaluation pipeline** that prevents code-injection vulnerabilities without sacrificing the full expressiveness of user-defined predicates.
5. **Empirical validation** on a real-world dataset of over 2.2 million rows with 100% correctness across 25 diverse test patterns and throughput reaching up to 12 918 rows/sec on a commodity laptop.

Chapter 4

Methodology

The `MATCH_RECOGNIZE` engine described in this thesis follows a modular, five-stage architecture designed specifically for SQL row-pattern recognition over in-memory data. The engine executes queries directly on `pandas` DataFrames [monierashraf2025rowmatch] while preserving the full semantics prescribed by the SQL:2016 standard [iso2016, ISO2021, melton2017sqlrpr]. The pipeline comprises five consecutive phases: *parsing*, *semantic validation*, *logical plan construction*, *pattern compilation*, and *execution* (Fig. ??).

In the **parsing** phase the SQL query is transformed into a typed Abstract Syntax Tree (AST) that captures both the outer `SELECT/FROM` structure and all eight `MATCH_RECOGNIZE` sub-clauses [tarazona2021antlr, parr2013definitive]. **Semantic validation** then verifies schema references, variable definitions, predicate syntax, skip policies, and output modes, and applies SQL:2016 compliance rules covering navigation-function usage, measure semantics, and edge cases such as empty matches and partition boundaries.

The validated description is translated into a **logical plan** composed of modular operators for partitioning, sorting, matching, measuring, and skipping rows. From this plan the system constructs a Nondeterministic Finite Automaton (NFA) with guarded transitions that encode quantifiers, alternation, and `PERMUTE` constructs [salomaa1996nfa]. The NFA is built via Thompson’s construction extended for SQL:2016 semantics: `DEFINE` predicates are embedded as transition guards, ϵ -transitions are minimised, and subset construction with state deduplication is applied to limit state-space growth.

Finally, **execution** processes each ordered partition using the compiled automaton. Matches are materialised with the correct `RUNNING/FINAL` context semantics. The evaluation engine maintains per-match variable bindings, applies navigation functions (`FIRST`, `LAST`, `PREV`, `NEXT`) with partition-boundary awareness, and computes aggregates (`COUNT`, `SUM`, `AVG`) across matched rows, delivering results as standard `pandas` DataFrames.

The remainder of this chapter elaborates each phase in turn. Section 4.1 describes the SQL parsing pipeline. Section 4.2 establishes the finite-automata foundations. Section 4.3 presents the Pandas execution model. Section 4.4 details the performance-optimisation layer.

4.1 SQL Parsing

Transforming a `MATCH_RECOGNIZE` query from plain text into an executable representation requires a parsing pipeline that handles both standard SQL syntax and the extended row-pattern sub-language. This section explains parsing fundamentals, surveys existing SQL parsers and parser generators, motivates the design choices made in this implementation, and describes the resulting parser in detail.

4.1.1 What is Parsing?

Parsing refers to the analysis of an input string according to the rules of a formal grammar and its transformation into a machine-processable representation [mogensen2024compiler]. In essence, the parser examines a human-written query, determines which grammar rules govern its syntactic construction, and produces a tree data structure that reflects that derivation.

Parsing in this engine proceeds in two sequential stages: *lexical analysis* and *syntactic analysis* [5].

Lexical analysis. The raw character stream is segmented into a sequence of predefined, meaningful units called *tokens*. Keywords (`SELECT`, `MATCH_RECOGNIZE`, `PATTERN`, ...), identifiers, string and numeric literals, and punctuation are all examples of tokens. Whitespace and comments are discarded at this stage; only the ordered token sequence is passed to the next stage.

Syntactic analysis. The token stream is matched against the grammar rules of the language, and a valid input is translated into a tree representation. A valid input produces a *parse tree* (also called a *concrete syntax tree*) whose internal nodes correspond to grammar rules and whose leaves correspond to individual tokens. An invalid input causes the parser to report an error, indicating precisely where the text violated the grammar.

The two-stage design separates concerns cleanly: the lexer handles regular-language aspects (token shapes), while the parser handles context-free-language aspects (phrase structure). Together they convert a character string into a structured tree that subsequent compiler phases can traverse and transform.

4.1.2 Parsing SQL: Existing Solutions

Several Python libraries offer SQL parsing out of the box. `sqlparse` performs tokenisation and basic structural splitting (statements, clauses) but produces only a shallow token stream, not a typed AST, and does not model `MATCH_RECOGNIZE` at all. `sqlglot` is a more complete SQL transpiler that supports multiple SQL dialects and exposes a typed AST; however, its `MATCH_RECOGNIZE` support is partial and does not cover the full SQL:2016 pattern sub-language (e.g. exclusions, anchors, `PERMUTE`, bounded quantifiers). `mo_sql_parsing` converts SQL to JSON but similarly provides no coverage for row pattern constructs.

A widely used baseline for conventional SQL parsing is the **PostgreSQL parser**, which has been extracted from the PostgreSQL source code and released as a standalone C library via the `libpg_query` project. Built on top of `libpg_query`, the Python library `pglast` wraps the original C AST nodes as typed Python classes and provides functionality to reconstruct SQL text from an AST. Listing 4.1 shows a minimal usage example: the call to `parse_sql` returns a tree of typed node objects, and `RawStream` serialises it back to SQL.

```
1 from pglast import parse_sql
2 from pglast.stream import RawStream
3
4 tree_root = parse_sql("SELECT 42;")
5 print(tree_root)           # typed AST node objects
6 print(RawStream()(tree_root)) # SELECT 42
```

Listing 4.1: Parsing a SQL statement with `pglast` and reconstructing SQL text

Although `pglast` cannot parse `MATCH_RECOGNIZE`, it is useful for parsing the SQL sub-expressions that appear inside `DEFINE` predicates and `MEASURES` clauses (see Section 4.1.11). The absence of any library that handles the full SQL:2016 row-pattern sub-language made a custom parser necessary for this project.

4.1.3 Grammars

A grammar formally describes the syntax of a language [5]. The elementary constituents of a grammar are:

- **Terminal symbols** (tokens) — the elementary, indivisible symbols of the language. In this thesis the terms *terminal* and *token* are used interchangeably.
- **Non-terminals** — symbols that represent sets of strings composed of terminals, used to name syntactic constructs (e.g. *expression*, *pattern*, *clause*).

- **Production rules** — rewriting rules of the form $N \rightarrow \alpha$, where N is a non-terminal and α is a sequence of terminals and/or non-terminals. Each rule describes one way in which a particular construct of the language may be written [wagenknecht2022formal].

A grammar is specified as a list of production rules. Because the left-hand side of each rule contains exactly one non-terminal, the grammar is *context-free*: the applicable rules do not depend on the surrounding context [5]. For example, a grammar for one-or-more occurrences of the letter a can be written:

$$S \rightarrow a \quad | \quad aS$$

This form is also known as *Backus–Naur Form* (BNF). The *Extended* variant (EBNF) adds syntactic sugar for optional elements ($[]$), repetition ($\{ \}$), and grouping ($()$), making large grammars more readable. For example, the `PARTITION BY` clause can be expressed in EBNF as:

```
partitionBy : 'PARTITION' 'BY' expr (',' expr)* ;
```

SQL is a large context-free language; the `MATCH_RECOGNIZE` sub-language adds several hundred additional productions for pattern terms, quantifiers, alternation, grouping, anchors, exclusions, `PERMUTE`, and the eight sub-clauses.

4.1.4 Our Parsing Needs

We now examine exactly what must be parsed and what the output should look like. Listing 4.2 shows the full syntax of a query containing a `MATCH_RECOGNIZE` clause as supported by this implementation.

```

1 SELECT <expression> [, ...]
2 FROM   <table>
3 MATCH_RECOGNIZE (
4     [ PARTITION BY <column> [, ...] ]
5     [ ORDER BY <column> [ASC|DESC] [, ...] ]
6     [ MEASURES <expression> [AS] <alias> [, ...] ]
7     [ ONE ROW PER MATCH
8     | ALL ROWS PER MATCH
9     | SHOW EMPTY MATCHES
10    | OMIT EMPTY MATCHES
11    | WITH UNMATCHED ROWS ] ]
12 [ AFTER MATCH SKIP
13   { PAST LAST ROW
14   | TO NEXT ROW
15   | TO FIRST <variable>
```

```

16 | TO LAST <variable> } ]
17 PATTERN ( <row_pattern> )
18 [ SUBSET <subset_name> = ( <variable> [, ...] ) [, ...] ]
19 DEFINE <variable> AS <condition> [, ...]
20 ) AS <alias>
21 <rest>;

```

Listing 4.2: Syntax of a `MATCH_RECOGNIZE` query

`MATCH_RECOGNIZE` is an optional sub-clause of the `FROM` clause. For simplicity, the implementation is restricted to a single top-level `SELECT` statement: no Common Table Expression (CTE), `VIEW` definition, or inline sub-query nesting is supported. Similarly, the trailing `<rest>` part must not include set operations such as `UNION`, `INTERSECT`, or `EXCEPT`. These restrictions are sufficient to cover the full SQL:2016 row-pattern use-case within a Pandas-based execution engine.

The parser must extract all eight optional and mandatory sub-clauses of `MATCH_RECOGNIZE`—`PARTITION BY`, `ORDER BY`, `MEASURES`, `rows-per-match`, `after-match-skip`, `PATTERN`, `SUBSET`, and `DEFINE`—and deliver them to subsequent pipeline stages through a well-defined AST interface, described next.

The `MATCH_RECOGNIZE` AST Interface

The AST exposed by the parser to downstream stages is defined by a hierarchy of typed Python `@dataclass` objects. The root node, `FullQueryAST`, holds the outer `SelectClause`, `FromClause`, and a single `MatchRecognizeClause` object that groups all eight sub-clause nodes. The interface contract is:

- Every node is immutable after construction.
- Each `MATCH_RECOGNIZE` sub-clause maps to a dedicated typed dataclass (`PartitionByClause`, `OrderByClause`, `MeasuresClause`, `RowsPerMatchClause`, `AfterMatchSkipClause`, `PatternClause`, `SubsetClause`, `DefineClause`), enabling the NFA compiler to dispatch on type rather than parsing raw strings.
- `DEFINE` predicates and `MEASURES` expressions are captured as plain strings during the first pass and re-evaluated on demand through a lightweight expression evaluator (Section 4.1.11), keeping the main grammar concise.
- `Measure` nodes carry computed flags (`is_classifier`, `is_match_number`) that allow the execution layer to route output columns without additional string inspection.

4.1.5 Parser Generators

Since productivity is a goal, the question arises whether the creation of a lexer and parser can be automated. The kind of software addressing this issue is called a *parser generator* [mogensen2024compiler]: it takes a grammar as input and produces the source code of a parser that understands the syntax defined by that grammar. The target language of the generated source code must be Python, since that is the language the engine is written in.

Candidates that support Python as a target language include:

- **PLY** (Python Lex-Yacc) — a Python implementation of the classic 1970 Lex/Yacc tools [ply_tool].
- **Lark** — described as “a modern approach, with a focus on ergonomics, performance and modularity” [lark_tool].
- **ANTLR4** — a parser generator “widely used to build languages, tools, and frameworks; from a grammar, ANTLR generates a parser that can build and walk parse trees” [6].

Table 4.1 compares the three tools across criteria extracted from their respective documentation.

Table 4.1: Comparison of Python-compatible parser generators

Feature	PLY (Lex/Y- acc)	Lark	ANTLR4
Algorithm	LALR(1)	LALR / Earley	Adaptive LL(*) [parr2014adaptive]
Grammar notation	BNF	EBNF	EBNF
Ambiguity handling	Operator precedence declarations	Rule priorities; Earley resolves via priority	Production order & semantic predicates
Left recursion	Yes	Yes (LALR mode)	Yes
Target language	Python native	Python native	Java runtime required to generate Python parser
AST / parse tree	No	Yes (parse tree)	Yes (parse tree)
Tree visualiser	No	Dot output	Java GUI (antlr4-tools)
Traversal scaffolding	No	Transformer class; methods written manually	Listener & visitor generated automatically

Due to the manageable size and complexity of the `MATCH_RECOGNIZE` statement, any of the three tools would likely produce a satisfactory result. Nevertheless, two

reasons motivated the choice of ANTLR4. First, its parse-tree visualiser (via the `antlr4-tools` Java GUI) greatly aids debugging grammar rules. Second, and more crucially, ANTLR4 automatically generates a traversal skeleton—both listener and visitor base classes—for the parse tree. Because the raw parse tree must be transformed into the typed AST described in Section 4.1.4, a generated visitor base class eliminates manual tree-traversal boilerplate. An empirical study further demonstrates significantly higher performance and maintainability when using ANTLR compared to Lex/Yacc [`antlr_performance`].

In this thesis the workflow combining ANTLR4 with the typed `MATCH_RECOGNIZE` AST interface is evaluated for its suitability in parsing the full `MATCH_RECOGNIZE` statement.

4.1.6 Alternative Methods

Two non-generator approaches were considered and rejected.

Hand-written recursive descent. A recursive descent parser is straightforward to implement for small grammars: each non-terminal becomes a Python method that calls other methods for sub-rules. However, SQL’s grammar contains hundreds of productions; maintaining a hand-written parser as the grammar evolves is error-prone. The approach was ruled out on maintainability grounds.

Regex-based string extraction. Regular expressions cannot model recursive structures such as nested grouping $((A+B)*|C)$ or balanced parentheses, so they cannot serve as a complete parser. They can pre-screen queries or extract simple fragments, but any production use requires a proper parser as the primary mechanism.

4.1.7 ANTLR4

ANTLR4 (ANother Tool for Language Recognition, version 4) is a parser generator that produces top-down, Adaptive LL(*) recursive-descent parsers from context-free grammar specifications written in the ANTLR meta-language [`parr2014adaptive`, `antlr_website`, `parr_antlr_docs`, 6]. Given a `.g4` grammar file the tool generates a lexer, a parser, and listener/visitor scaffolding in the target language (Python in this project).

The implementation extends Trino’s production SQL grammar (`TrinoLexer.g4` / `TrinoParser.g4`) with specialised rules for every sub-clause of `MATCH_RECOGNIZE`, as shown in the excerpt in Listing 4.3. A custom error listener intercepts syntax errors and reports them with precise line/column positions and contextual suggestions.

4.1.8 Implementation of the Parser

Designing the Grammar

The implementation uses the production Trino SQL grammar obtained from the ANTLR community grammars project (`TrinoLexer.g4` / `TrinoParser.g4`, approximately 1 124 lines in total) [`antlr_trino_sql`]. This grammar natively contains the `patternRecognition` rule that covers all eight `MATCH_RECOGNIZE` sub-clauses; no modifications to the grammar file were required. Listing 4.3 shows the key productions.

```
1 patternRecognition
2     : aliasedRelation (
3         MATCH_RECOGNIZE '('
4             (PARTITION BY expression (',' expression)*)?
5             (ORDER BY sortItem (',' sortItem)*)?
6             (MEASURES measureDefinition (',' measureDefinition)*)?
7             rowsPerMatch?
8             (AFTER MATCH skipTo)?
9             (INITIAL | SEEK)?
10            PATTERN '(' rowPattern ')'
11            (SUBSET subsetDefinition (',' subsetDefinition)*)?
12            (DEFINE variableDefinition (',' variableDefinition)*)?
13            ')' (AS? identifier columnAliases?)?
14        )?
15    ;
16
17 measureDefinition : expression AS identifier ;
18
19 rowsPerMatch
20     : ONE ROW PER MATCH
21     | ALL ROWS PER MATCH emptyMatchHandling?
22     ;
23
24 skipTo
25     : SKIP TO NEXT ROW
26     | SKIP TO (FIRST | LAST)? identifier
27     | SKIP PAST LAST ROW
28     ;
29
30 rowPattern
31     : rowPattern rowPattern # patternConcatenation
32     | rowPattern '|' rowPattern # patternAlternation
33     | patternPrimary patternQuantifier? # quantifiedPrimary
34     ;
35
36 patternPrimary
```

```

37      : identifier                                #
        patternVariable
38      | '(' rowPattern ')'                        #
        groupedPattern
39      | '{-' rowPattern '-}'                    #
        excludedPattern
40      | PERMUTE '(' rowPattern (',' rowPattern)* ')' #
        patternPermutation
41      ;

```

Listing 4.3: Key productions of the `patternRecognition` rule in `TrinoParser.g4`

Handling Expressions as Strings

DEFINE predicates and MEASURES expressions contain arbitrary column references, navigation function calls, arithmetic, and comparisons. Parsing these into a fully typed expression tree requires the grammar to enumerate every possible SQL operator and function name. An alternative—adopted here—is to capture such expressions as structured token sequences during parsing, then re-parse them through a lightweight expression parser on first use. This two-pass strategy keeps the main grammar concise while still producing a typed AST for safe evaluation (see Section 4.1.11).

Parse Tree vs. Abstract Syntax Tree

Parse trees produced by ANTLR4 contain many structural nodes (keywords, punctuation) that are unnecessary for execution. A visitor traverses the parse tree and produces a compact *Abstract Syntax Tree* (AST) whose node types correspond directly to semantic constructs [jones2003ast, 5]. The key node types are summarised in Table 4.2.

Table 4.2: Principal AST node types

Node Type	Represents
FullQueryAST	Root of the full query (SELECT + FROM + MR)
MatchRecognizeClause	Root of the MATCH_RECOGNIZE sub-clause
PartitionByClause	PARTITION BY column list
OrderByClause	ORDER BY sort items (SortItem)
MeasuresClause	MEASURES list (Measure nodes)
RowsPerMatchClause	ONE / ALL ROWS PER MATCH mode
AfterMatchSkipClause	AFTER MATCH SKIP policy
PatternClause	PATTERN string with tokenizer
SubsetClause	SUBSET variable definitions
DefineClause	DEFINE list (Define nodes)

The AST is validated for semantic correctness before pattern compilation: undefined variable references in `DEFINE` predicates, circular definitions, and illegal quantifier ranges are all detected and reported at this stage.

4.1.9 Visitor Implementation

ANTLR4 generates an abstract visitor base class from the grammar [robson2022designpatterns]. The parsing module is structured as two cooperating visitor classes built on the generated `TrinoParserVisitor` base.

MatchRecognizeExtractor. The inner visitor overrides `visitPatternRecognition` and extracts all eight sub-clauses in strict grammar order: `PARTITION BY` column expressions, `ORDER BY` sort items with direction and nulls handling, `MEASURES` definitions (handling `RUNNING/FINAL` prefix and alias via a dedicated `smart_split()` helper that correctly traverses nested parentheses and quoted identifiers), rows-per-match mode, after-match-skip policy, the raw pattern string, `SUBSET` definitions, and `DEFINE` variable-condition pairs. After extraction, a four-step validation chain is applied: `validate_clauses` (ensures `PATTERN` and `DEFINE` are both present when required); `validate_pattern_clause` (checks balanced parentheses and flags conflicts such as pattern exclusion combined with `WITH UNMATCHED ROWS`); `validate_function_usage` (permits only the whitelisted navigation and aggregate functions—`FIRST`, `LAST`, `PREV`, `NEXT`, `COUNT`, `SUM_IF`, `AVG_IF`, `COUNT_IF`—inside `MEASURES`); and `validate_pattern_variables_defined` (verifies that every variable referenced in the pattern appears in `DEFINE` or `SUBSET`, and detects table-qualified references that would be ambiguous at execution time).

FullQueryExtractor. The outer visitor handles the top-level `singleStatement` rule. It extracts the outer `SELECT` column list, `FROM` table reference, and any trailing `ORDER BY` clause via regex, then locates the `PatternRecognitionContext` node through a recursive tree walk and delegates to `MatchRecognizeExtractor`.

Public API. The entry point for all callers is `parse_full_query(query: str) → FullQueryA`. The parser implementation is validated through a structured test suite located in the `tests/` directory. Tests are organised into three layers: unit tests for individual extractor methods, integration tests that exercise the full `parse_full_query` pipeline, and SQL:2016 compliance tests that compare output against Trino’s reference behaviour.

Unit tests. Each of the eight `extract_*` methods is tested in isolation against known SQL fragments. For example, `extract_measures` is fed a `MEASURES`

token stream containing RUNNING, FINAL, and undecorated expressions, and the resulting Measure objects are asserted for correct alias, expression text, and semantic flags (explicit_semantics, is_classifier, is_match_number). The validation chain is also tested adversarially: supplying an undefined pattern variable, a prohibited function in MEASURES, or a PATTERN clause without a corresponding DEFINE all trigger a ParserError carrying the expected line and column information.

Integration tests. The file test_match_recognize.py mirrors Trino's TestRowPattern and exercises twelve scenarios end-to-end: simple queries, pattern quantifiers (*, +, ?, bounded {n,m}), pattern exclusion syntax ({-...-}), all four after-match-skip modes, one-row and all-rows output modes, CLASSIFIER() and MATCH_NUMBER() special columns, all four navigation functions, partitioned and ordered input, SUBSET aliasing, and RUNNING/FINAL semantics.

SQL:2016 compliance tests. test_sql2016_compliance.py contains seven test classes targeted specifically at standard conformance: pattern syntax, navigation functions, special aggregate functions, partition boundary handling, subset variable resolution, output-mode semantics, and all six skip policies. Empty-match handling is covered separately in test_empty_matches.py with nine test cases including SHOW EMPTY MATCHES, OMIT EMPTY MATCHES, WITH UNMATCHED ROWS, and combinations with alternation and partitioning.

Edge-case tests. Additional test files cover IN () predicate preprocessing (test_in_predicate.py), IS NULL handling (test_is_null_handling.py), anchored patterns (test_anchor_patterns.py), PERMUTE patterns (test_permute_patterns.py), back-references (test_back_reference.py), pattern cache correctness and thread safety (test_pattern_cache.py), and production-scale aggregation accuracy (test_production_aggregates.py).

4.1.10 Discussion of the Parser

The grammar-driven approach provides several practical advantages: the language specification remains in a single readable .g4 file; ANTLR4's error recovery inserts synthetic tokens to continue parsing past a local error, collecting multiple diagnostics in one pass; and the generated visitor scaffolding eliminates manual tree traversal boilerplate.

Limits

Two limitations are known. First, some advanced SQL:2016 expression constructs (e.g. recursive WITH RECURSIVE inside a DEFINE predicate) are not supported; the expression sub-parser recognises only the constructs needed for MATCH_RECOGNIZE evaluation. Second, because the grammar extends Trino's dialect, vendor-specific extensions of other systems (Oracle RUNNING keyword placement differences, Snowflake aggregate variants) require minor grammar adjustments if cross-dialect compatibility is needed.

4.1.11 Safe SQL Expression Evaluation

DEFINE predicates and MEASURES expressions are user-supplied strings that must be evaluated as Python code at runtime. Directly calling `eval()` on arbitrary user input creates a code-injection vulnerability. The implementation instead uses Python's `ast` module to parse and validate expressions before execution, as shown in Listing 4.4.

```
1 import ast
2
3 def safe_evaluate(sql_expr, row_data, context):
4     # 1. Translate SQL syntax to Python equivalents
5     py_expr = translate_sql_to_python(sql_expr)
6
7     # 2. Parse into a Python AST (expression mode only)
8     tree = ast.parse(py_expr, mode='eval')
9
10    # 3. Whitelist-validate: allow comparisons, arithmetic,
11    #     boolean ops, literals, subscripts -- forbid imports,
12    #     function calls (except navigation functions),
13    assignments
14    ASTSecurityValidator().visit(tree)
15
16    # 4. Compile and execute in a controlled namespace
17    code = compile(tree, '<expr>', 'eval')
18    return eval(code, {'__builtins__': {}},
19                  {'row': row_data, **context})
```

Listing 4.4: Safe SQL expression evaluation using AST whitelisting

The controlled namespace exposes only the current row dictionary and the whitelisted navigation functions (PREV, NEXT, FIRST, LAST), providing a safe sandbox with no access to the file system, network, or interpreter

internals.

4.2 Theoretical Foundations: Finite Automata

Pattern matching in MATCH_RECOGNIZE is fundamentally a problem of sequence recognition. A sequence of data rows, each labelled with the pattern variable(s) it satisfies, constitutes a string over the alphabet of pattern variables. The problem of determining whether this string is accepted by the pattern is exactly the problem solved by finite automata [3].

4.2.1 Non-deterministic Finite Automata (NFA)

A *Non-deterministic Finite Automaton* (NFA) is formally defined as a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where:

- Q is a finite set of *states*;
- Σ is the *input alphabet* (pattern variables in our setting);
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$ is the *transition function*, mapping each state and symbol (or the empty string ε) to a *set* of successor states;
- $q_0 \in Q$ is the *initial state*; and
- $F \subseteq Q$ is the set of *accepting states*.

NFAs naturally model alternation and optional constructs owing to their ability to be in multiple states simultaneously and to use ε -transitions (transitions that consume no input).

Figure 4.1 shows the NFA for pattern $A^+ B?$.

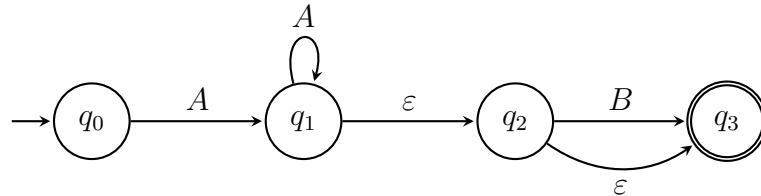


Figure 4.1: NFA for the pattern $A^+ B?$ illustrating ε -transitions and non-determinism

4.2.2 Thompson's Construction

Thompson's construction algorithm [4] converts a regular expression into an NFA in linear time $O(n)$, where n is the length of the expression. The algorithm proceeds recursively over the expression structure according to the rules in Algorithm 1.

Algorithm 1 Thompson's NFA Construction

```

1: procedure BUILD( $e$ )
2:   if  $e = \varepsilon$  then
3:     return NFA with one  $\varepsilon$ -transition from start to accept
4:   else if  $e = a \in \Sigma$  then
5:     return NFA with one  $a$ -labelled transition from start to accept
6:   else if  $e = r \cdot s$  (concatenation) then
7:      $N_r \leftarrow \text{BUILD}(r)$ ;  $N_s \leftarrow \text{BUILD}(s)$ 
8:     Merge accept state of  $N_r$  with start state of  $N_s$ 
9:     return combined NFA
10:  else if  $e = r \mid s$  (alternation) then
11:     $N_r \leftarrow \text{BUILD}(r)$ ;  $N_s \leftarrow \text{BUILD}(s)$ 
12:    Add new start  $q_{\text{new}}$  with  $\varepsilon$ -transitions to  $q_0^{N_r}$  and  $q_0^{N_s}$ 
13:    Add new accept  $f_{\text{new}}$  with  $\varepsilon$ -transitions from  $F^{N_r}$  and  $F^{N_s}$ 
14:    return combined NFA
15:  else if  $e = r^*$  (Kleene star) then
16:     $N_r \leftarrow \text{BUILD}(r)$ 
17:    Add  $\varepsilon$ -transition from start to accept (zero repetitions)
18:    Add  $\varepsilon$ -transition from  $F^{N_r}$  back to  $q_0^{N_r}$  (repetition)
19:    return modified NFA
20:  else if  $e = r^{\{n,m\}}$  (bounded quantifier) then
21:    Construct  $n$  mandatory copies followed by  $(m - n)$  optional copies
22:    return combined NFA
23:  end if
24: end procedure

```

The resulting NFA has at most $2n$ states and is constructed in $O(n)$ time and space, making it suitable for patterns of arbitrary complexity [5].

4.2.3 Pattern Tokenisation

Before an NFA can be constructed the raw pattern string extracted from the SQL query must be converted into a structured, traversable representation. The `tokenize_pattern()` function in `src/matcher/pattern_tokenizer.py` performs this step in a single left-to-right scan, producing a flat list of `PatternToken` objects—the *linearised* form of the pattern's syntax tree.

PatternToken structure. Each `PatternToken` is a Python dataclass with the following key fields:

- **type** - a `PatternTokenType` enum value determining which NFA fragment builder is invoked (see Table 4.3);
- **value** - the raw textual value extracted from the pattern string (variable name, operator character, etc.);
- **quantifier** - optional quantifier string ('*', '+', '?', or '{n,m}') copied from the immediately following quantifier when present; defaults to None;
- **greedy** - boolean flag indicating greedy (True, the default) or reluctant (False) quantification, set when a trailing ? follows the quantifier;
- **metadata** - a dictionary carrying type-specific structured data (e.g. a variables list for `PERMUTE` tokens, an `excluded_variables` list for `EXCLUSION` tokens); and
- **priority** - an integer priority inherited from the token's position in the stream, propagated to NFA transitions to enforce greedy/leftmost semantics.

Table 4.3: The thirteen `PatternTokenType` values and their roles in the pattern string

Token type	Role in pattern
LITERAL	A pattern variable name (A, UP, STRT, etc.)
ALTERNATION	The operator
PERMUTE	A complete <code>PERMUTE(...)</code> construct carrying its variable list in <code>metadata</code>
GROUP_START	An opening parenthesis (
GROUP_END	A closing parenthesis)
ANCHOR_START	The start-of-partition anchor ^
ANCHOR_END	The end-of-partition anchor \$
EXCLUSION	A complete {- ...-} exclusion block
EXCLUSION_START	Opening delimiter {-
EXCLUSION_END	Closing delimiter -}
QUANTIFIER	A standalone quantifier node
SUBSET	A SUBSET variable reference
SEQUENCE	A composite sequence node

Tokenisation algorithm. The internal function `_tokenize_pattern_internal()` scans the pattern string character by character, dispatching on the current position:

1. Whitespace is skipped unconditionally.
2. The keyword PERMUTE (case-insensitive) triggers `_parse_permute_pattern()`, which collects the comma-separated variable list inside the following parentheses and returns a single PERMUTE token whose `metadata['variables']` holds the parsed variable tokens.
3. The two-character sequence `{-` triggers `_parse_exclusion_pattern()`, which reads to the matching `-}` (tracking nesting depth) and returns an EXCLUSION token.
4. The structural characters `(, ), |, ^,` and `$` each produce a single-character token of the corresponding type.
5. Quantifier characters `*, +, ?, and {` are parsed by `parse_quantifier_at()` and *attached* to the preceding token as its quantifier and greedy fields-no separate token is inserted.
6. Identifier characters (`[A-Za-z0-9_]`) are accumulated into a LITERAL token.

After the scan, `_optimize_token_sequence()` makes a conservative pass that removes structural redundancies while preserving the one-variable-per-token invariant that the NFA builder requires: adjacent LITERAL tokens that look like pattern variable names are deliberately *not* merged.

Quantifier normalisation. Quantifiers are stored as attributes of the token they modify rather than as separate tokens. The helper `parse_quantifier()` normalises all surface forms to the triple $(min, max, greedy)$:

<code>?</code>	$\rightarrow$	$(0, 1, True)$
<code>*</code>	$\rightarrow$	$(0, \infty, True)$
<code>+</code>	$\rightarrow$	$(1, \infty, True)$
<code>{n}</code>	$\rightarrow$	$(n, n, True)$
<code>{n,m}</code>	$\rightarrow$	$(n, m, True)$
<code>append ? to any</code>	$\rightarrow$	$greedy = False$

This triple is consumed directly by `NFABuilder._apply_quantifier()` during NFA construction.

4.2.4 NFA Construction from Tokens

The `NFABuilder` class in `src/matcher/automata.py` implements the *visitor* pattern over the token list. Its central method `_process_sequence()` iterates

the token stream and dispatches on each token's type field to invoke the appropriate NFA-fragment builder-mirroring the recursive tree-traversal structure of classical Thompson's construction but applied to the flat linearised token representation rather than an explicit parse tree.

Overall build sequence. The public entry point `NFABuilder.build()` executes the following steps:

1. Check the LRU pattern cache (keyed on an MD5 hash of the token list and DEFINE context) for a previously compiled NFA. A cache hit returns a deep clone of the stored NFA to prevent state sharing.
2. On a cache miss call `_build_nfa_internal()`, which:
 - (a) validates anchor semantics (e.g. rejecting patterns such as $A^\wedge$ where a start anchor follows a literal);
 - (b) creates a global start state q_{start} and accept state q_{accept} ;
 - (c) delegates to `_process_sequence()` for the pattern body; and
 - (d) connects the pattern fragment to the global states via ε -transitions.
3. Apply `NFA.optimize()` to remove unreachable states, deduplicate transitions, and pre-compute DFA hot-paths.
4. Cache the compiled NFA and return it.

Token-to-NFA Fragment Mapping

Table 4.4 summarises how the visitor translates each token type into NFA structure.

Quantifier Expansion

`_apply_quantifier(start, end, min, max, greedy)` wraps a fragment (s, e) in outer envelope states (q_s, q_e) and adds ε -transitions accordingly:

- `?` ($\min = 0, \max = 1$): adds $\varepsilon(q_s \rightarrow s)$ for the match case and $\varepsilon(q_s \rightarrow q_e)$ for the skip case.
- `*` ($\min = 0, \max = \infty$): adds $\varepsilon(q_s \rightarrow s)$, $\varepsilon(e \rightarrow q_e)$ (exit), $\varepsilon(q_s \rightarrow q_e)$ (skip), and $\varepsilon(e \rightarrow s)$ (loop for repetition).
- `+` ($\min = 1, \max = \infty$): as `*` but without the skip transition, enforcing at least one match.

Table 4.4: Token types and their NFA fragment builders in `NFABuilder._process_sequence()`

Token type	NFA construction rule
LITERAL	<code>create_var_states()</code> produces a two-state fragment $q_s \xrightarrow{C(v)} q_e$, where $C(v)$ is the callable compiled from the <code>DEFINE</code> predicate for variable v ; the token's <code>quantifier</code> (if any) is applied by <code>_apply_quantifier()</code>
ALTERNATION	The left branch is finalised; the right branch is processed by a recursive call to <code>_process_sequence()</code> ; a fresh start state fans out with ε -transitions to both branch heads, and a fresh end state collects both branch tails
GROUP_START	Triggers a recursive call to <code>_process_sequence()</code> that reads until the matching <code>GROUP_END</code> ; the group quantifier (if any) is applied to the resulting fragment
PERMUTE	<code>_process_permute()</code> expands all $n!$ orderings of the n variables and builds a union of $n!$ concatenation fragments under a shared start/end pair
EXCLUSION	<code>_process_exclusion_token()</code> builds the contained sub-pattern normally but marks every state in the range with <code>is_excluded = True</code> ; the matcher filters excluded variables from output at match time
ANCHOR_START/END	Creates a single anchor state with <code>is_anchor = True</code> and the corresponding <code>anchor_type</code> ; partition-boundary semantics are enforced by the matcher

- $\{n, m\}$: constructs n mandatory copies of the fragment in series (connected by ε -transitions), followed by $(m-n)$ optional copies each of which carries an ε -bypass to q_e -the standard elimination-of-quantifier reduction.
- **Reluctant** (greedy = False): the same ε structure is used, but the loop-back and extension transitions are annotated with a *higher* numeric priority value so the epsilon-closure traversal prefers the exit path, yielding shorter matches.

PERMUTE Expansion

The `PERMUTE( $v_1, v_2, \dots, v_n$ )` construct is semantically equivalent to the alternation of all $n!$ orderings of its arguments. `_process_permute()` proceeds in three phases:

1. **Complexity guard.** Patterns with more than eight variables are rejected immediately (raising `RuntimeError`) because $8! = 40\,320$ is the largest feasible inline expansion; patterns estimated to produce more than 50 000 combinations are redirected to a conservative fallback via `_process_pe`.
2. **Permutation generation.** `ProductionPermuteHandler.expand_permutation()` is called, which uses `itertools.permutations` for $n \leq 6$ and a memory-efficient streaming algorithm for larger n . Results are memoised in a thread-safe LRU cache keyed on the sorted variable tuple.
3. **NFA assembly.** For each permutation $\sigma = (\sigma_1, \dots, \sigma_n)$, the builder constructs a concatenation fragment by calling `create_var_states()` for each σ_i and connecting the fragments with ε -transitions. All $n!$ concatenation fragments are then united under a common fork/merge pair with the same ε -alternation structure used for the `|` operator.

4.2.5 Properties of the Constructed NFA

The NFA returned by `NFABuilder.build()` satisfies the following formal properties:

1. **Linear size.** At most two states per LITERAL token (the transition's source and target) plus a bounded constant of intermediate states per structural token (alternation fork/merge, quantifier envelope, etc.). Total state count is $O(n)$ in token-list length n .
2. **Single accept state.** Exactly one accept state q_{accept} exists; all fragment tails are connected to it via ε -transitions.

3. **Priority-annotated ε -transitions.** Every ε -transition carries an integer priority stored in the `epsilon_priorities` dictionary on its source `NFAState`. Lower numeric values denote higher priority. For greedy quantifiers the loop-back ε has lower priority than the exit ε , so the ε -closure computation (which sorts targets by ascending priority) extends the match before terminating. Reluctant quantifiers reverse the ordering. These priorities are propagated through subset construction so that each DFA transition inherits the minimum (highest-priority) value of the contributing NFA transitions, implementing SQL:2016 greedy and leftmost-match semantics without modifying the DFA construction algorithm.
4. **Thread safety.** Each `NFAState` and the NFA object itself are guarded by a `threading.RLock()`. Epsilon-closure results are cached in a bounded dictionary (capped at 1000 entries) accessed under the same lock, enabling safe concurrent queries by multiple partition workers.
5. **Structural validation.** `NFA.validate()` verifies that (i) all transition targets are valid state indices, (ii) the accept state is reachable from the start state, and (iii) PERMUTE metadata is internally consistent before the NFA is returned to the caller.

4.2.6 Worked Example: Pattern `A B+ C?`

Consider the pattern `A B+ C?` with three pattern variables `A`, `B`, `C` each associated with an arbitrary Boolean `DEFINE` predicate. The tokeniser produces the flat list:

```
[LITERAL('A'), LITERAL('B', quantifier='+'), LITERAL('C',
                                quantifier='?')]
```

The `NFABuilder` then processes this list as follows.

1. A global start state q_0 and accept state q_a are created; q_a is marked `is_accept = True`.
2. **LITERAL A (no quantifier).** `create_var_states` allocates q_1 and q_2 ; adds transition $q_1 \xrightarrow{C(A)} q_2$. The visitor adds $\varepsilon(q_0 \rightarrow q_1)$ and advances current to q_2 .
3. **LITERAL B with quantifier +.** `create_var_states` allocates q_3, q_4 with $q_3 \xrightarrow{C(B)} q_4$. `_apply_quantifier( $q_3, q_4, 1, \infty, True$ )` allocates envelope states q_5, q_6 and adds: $\varepsilon(q_5 \rightarrow q_3)$, $\varepsilon(q_4 \rightarrow q_6)$ (exit), $\varepsilon(q_4 \rightarrow q_3)$ (loop, greedy). The visitor adds $\varepsilon(q_2 \rightarrow q_5)$ and advances to q_6 .

4. **LITERAL C with quantifier ?.** `create_var_states` allocates q_7, q_8 with $q_7 \xrightarrow{C(C)} q_8$. `_apply_quantifier( $q_7, q_8, 0, 1, True$ )` allocates q_9, q_{10} and adds: $\varepsilon(q_9 \rightarrow q_7)$ (take C), $\varepsilon(q_8 \rightarrow q_{10})$ (exit after C), $\varepsilon(q_9 \rightarrow q_{10})$ (skip C , bypass). The visitor adds $\varepsilon(q_6 \rightarrow q_9)$ and advances to q_{10} .
5. $\varepsilon(q_{10} \rightarrow q_a)$ connects the pattern's tail to the global accept state.

The resulting NFA (11 states: q_0 - q_{10} plus the accept state shared with q_{10} in this abbreviated numbering) accepts exactly the language $A \cdot B^+$. $C^?$: one row satisfying A , one or more rows satisfying B (the loop $q_4 \rightarrow q_3$ implements the $+$ greedily), and zero or one row satisfying C (the bypass $q_9 \rightarrow q_{10}$ implements the $?$).

After construction the NFA is passed to `DFABuilder.build()` for subset construction as described in the next section.

4.2.7 Deterministic Finite Automata (DFA)

A *Deterministic Finite Automaton* (DFA) is a special case of an NFA in which: (i) there are no ε -transitions, and (ii) for each state q and symbol a , $|\delta(q, a)| = 1$ (exactly one successor state). DFAs require $O(n)$ time to process an input of length n with no backtracking, making them ideal for row-by-row matching.

Subset Construction (NFA $\rightarrow$ DFA). The *subset construction* (powerset construction) algorithm converts an NFA to an equivalent DFA. Each DFA state corresponds to a *set* of NFA states. For a DFA state S and input symbol a :

$$\delta_{\text{DFA}}(S, a) = \varepsilon\text{-closure}\left(\bigcup_{q \in S} \delta_{\text{NFA}}(q, a)\right)$$

where $\varepsilon\text{-closure}(T)$ is the set of all NFA states reachable from any state in T by zero or more ε -transitions. Although the resulting DFA may have up to $2^{|Q|}$ states in the worst case, patterns arising from `MATCH_RECOGNIZE` syntax are structurally constrained and in practice yield DFAs with far fewer states.

DFA Minimisation. Hopcroft's algorithm [3] reduces a DFA to its canonical minimal form in $O(n \log n)$ time by iteratively partitioning states into equivalence classes of indistinguishable states. The implementation applies minimisation after subset construction to reduce memory footprint and improve matching speed.

State Prioritisation for SQL Semantics. SQL:2016 mandates *greedy* matching by default: the engine must prefer the longest possible match, and in the case of alternation, the leftmost alternative takes precedence. Standard DFA theory does not encode these preferences. The implementation addresses this by:

1. Assigning a *priority* to each NFA transition based on its position in the pattern and quantifier greediness.
2. Propagating priorities through subset construction so that each DFA transition inherits the minimum (highest-priority) value of the contributing NFA transitions.
3. Ordering outgoing DFA transitions by priority during match execution so that higher-priority paths are explored first.

4.3 Pandas DataFrame Execution Model

Pandas [7] is a Python library providing the DataFrame data structure: a two-dimensional, labelled table backed by typed NumPy arrays. Its in-memory, columnar storage enables vectorised operations over millions of rows.

The MATCH_RECOGNIZE engine adopts the following execution strategy over a DataFrame D :

1. **Partition.** Group D by the PARTITION BY columns using DataFrame.groupby(), producing a collection of independent sub-frames.
2. **Order.** Sort each partition by the ORDER BY columns using sort_values().
3. **Classify rows.** For each row, evaluate every DEFINE predicate and record which pattern variables the row satisfies.
4. **Simulate the DFA.** Feed the classified sequence into the compiled DFA row by row. On reaching an accepting state, record the matched row indices and the active variable assignments.
5. **Evaluate measures.** For each recorded match, evaluate the MEASURES expressions using the variable assignments and navigation functions.
6. **Assemble output.** Concatenate match results into the output DataFrame with the declared measure columns.

Navigation functions. During measure evaluation, navigation functions resolve to direct DataFrame row accesses:

- `FIRST(var.col)` $\rightarrow$ `df.iloc[match[var][0]]['col']`
- `LAST(var.col)` $\rightarrow$ `df.iloc[match[var][-1]]['col']`
- `PREV(col, k)` $\rightarrow$ `df.iloc[current_idx - k]['col']`
- `NEXT(col, k)` $\rightarrow$ `df.iloc[current_idx + k]['col']`

4.4 Performance Optimization Techniques

4.4.1 LRU Pattern Caching

Pattern compilation--tokenisation $\rightarrow$ NFA construction $\rightarrow$ DFA conversion $\rightarrow$ minimisation--is computationally expensive, especially for patterns with quantifiers and PERMUTE constructs. The implementation maintains a thread-safe *Least Recently Used* (LRU) cache that maps a hash of the pattern string and DEFINE predicates to the compiled DFA. Key configuration parameters (configurable via environment variables) are listed in Table 4.5.

Table 4.5: Pattern cache configuration defaults

Parameter	Default	Description
CACHE_SIZE_LIMIT	10 000 entries	Maximum cached patterns
CACHE_MEMORY_LIMIT_MB	500 MB	Maximum cache memory
CACHE_TTL_SECONDS	3 600 s	Entry time-to-live
CACHE_CLEAR_THRESHOLD_MB	400 MB	Proactive eviction threshold

4.4.2 Memory Management

Large-scale matching over millions of rows requires careful memory management. The implementation processes one partition at a time, accumulating matches in a fixed-size buffer before flushing results to the output DataFrame. Memory usage is monitored via psutil; garbage collection is triggered proactively when consumption approaches user-configured limits. Object pools reuse DFA state structures and transition objects across successive matches, reducing allocator pressure.

Chapter 5

Experimental Evaluation

5.1 Evaluation Methodology

[Describe evaluation methodology here.]

5.2 Dataset and Test Patterns

[Describe benchmark dataset and 25 test patterns here.]

5.3 Correctness Results

[Present correctness results here.]

5.4 Performance Results

[Present throughput and scalability results here.]

5.5 Comparison with Existing Systems

[Compare with Oracle, Trino, Flink, etc.]

Chapter 6

Conclusions and Future Work

6.1 Summary of Contributions

[Summarise contributions here.]

6.2 Limitations

[Discuss limitations here.]

6.3 Future Work

[Outline directions for future research here.]

Bibliography

- [1] ISO/IEC. *ISO/IEC 9075-2:2016 Information technology — Database languages — SQL — Part 2: Foundation (SQL/Foundation)*. International Standard. International Organization for Standardization, 2016.
- [2] Oracle Corporation. *Pattern Matching in Oracle Database 12c*. https://docs.oracle.com/database/121/DWHSG/pattern_match.htm. Accessed: 2024. 2013.
- [3] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3rd. Boston, MA, USA: Addison-Wesley, 2006. ISBN: 978-0-321-45536-9.
- [4] Ken Thompson. “Programming Techniques: Regular Expression Search Algorithm”. In: *Communications of the ACM* 11.6 (1968), pp. 419–422. DOI: 10.1145/363347.363387.
- [5] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd. Boston, MA, USA: Addison-Wesley, 2006. ISBN: 978-0-321-48681-3.
- [6] Terence Parr. *The Definitive ANTLR 4 Reference*. Lewisville, TX, USA: Pragmatic Bookshelf, 2013. ISBN: 978-1-934356-99-9.
- [7] Wes McKinney. “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a.
- [8] Alexander Alexandrov et al. “The Stratosphere Platform for Big Data Analytics”. In: *The VLDB Journal*. See also: Apache Flink CEP documentation <https://nightlies.apache.org/flink/flink-docs-stable/docs/libs/cep/>. 2014.
- [9] Trino Project. *Trino Documentation: Pattern Recognition in Window Functions*. <https://trino.io/docs/current/sql/match-recognize.html>. Accessed: 2024. 2023.